

还会保存一些额外的元数据，这些元数据用于辅助存储介质的管理。多个页组成一个块 (Block)，块大小多在 2MB 左右。多个块组成一个面 (Plane)。一些存储命令可以同时操作两个面中的数据。面进而组成带 (Die)，也被称为芯片 (Chip)。通常一个闪存存储介质会包含几个带，这些带共享一组数据总线和一些常用的信号控制线，但也各自拥有如使能 (Enable)、就绪 (Ready) 和忙碌 (Busy) 等信号。除了访问性能和存储结构之外，闪存存储的访问特性也与机械硬盘有较大不同。下面介绍闪存存储的主要特性。

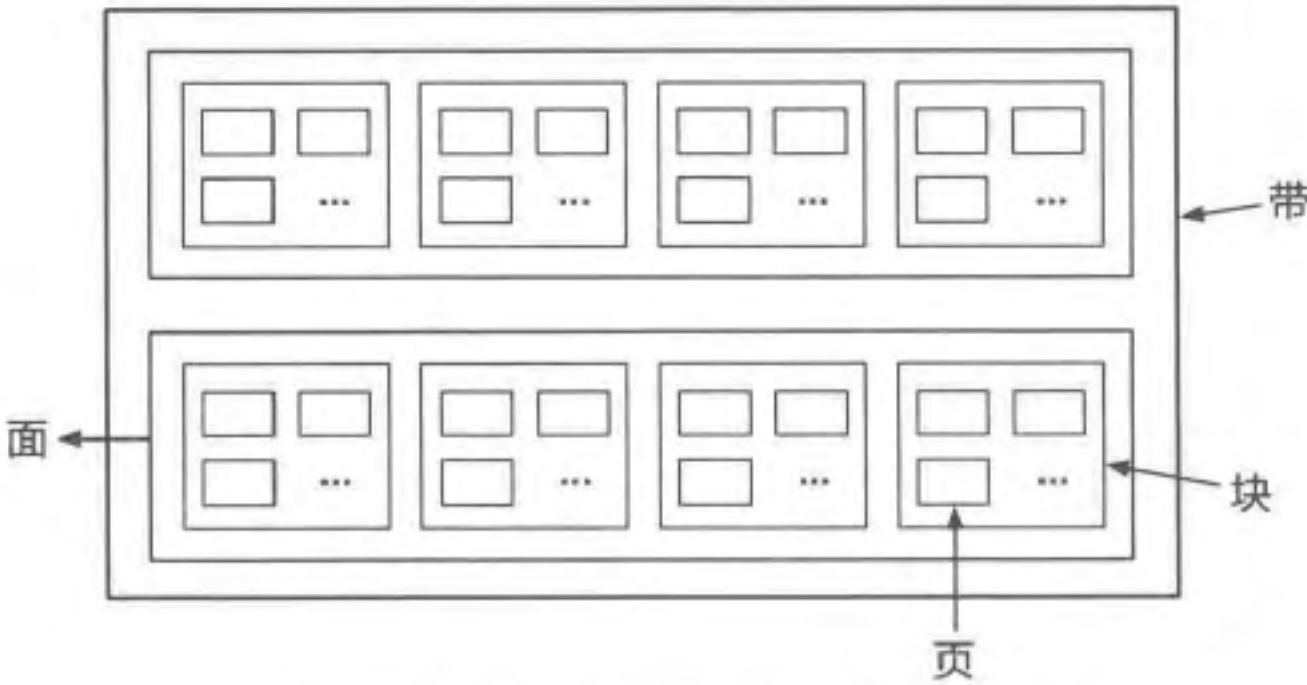


图 13.8 闪存存储的主要结构

写入与擦除循环。闪存存储的主要特点是已经写入的数据不能直接以覆盖的形式修改。如果想要修改，需要先执行擦除操作。在擦除时，闪存存储以块粒度进行擦除。例如，若需要修改闪存上的 1 个字节，需要将该字节所在闪存块——假设是 2MB——保存的数据先读入内存，然后将 2MB 的闪存块全部擦除，再把修改后的数据从内存写回闪存块。这里的写放大倍数高达 2 000 000 倍 (2MB/1 字节)。

磨损和耐久度。随着使用次数的增加，闪存存储介质的存储能力会逐渐下降，即产生磨损。闪存中的存储单元在进行大量写入和擦除后，会发生写穿 (Wear-out) 问题，无法继续用于准确地保存和读取数据。设备控制器中经常使用磨损均衡 (Wear Leveling) 技术，尽可能地让写入操作平均分摊在不同的存储单元中，使所有的存储单元尽可能同时老化，从而避免由于写穿的存储单元过多导致整个闪存无法使用。

固态硬盘

固态硬盘 (Solid State Drive, SSD) 由多块闪存 (Flash Package) 和相关控制器组成。图 13.9 展示了固态硬盘的结构。固态硬盘的控制器包括主机接口、闪存控制器、内存缓冲区等单元。主机接口负责与主机进行通信，通常是 NVMe、SATA、PCIe 等接口协议中的一种。对固态硬盘的管理和数据请求均需通过主机接口解码后交由闪存控制器进一步处理。请求完成后，再由主机接口单元按照对应通信协议进行编码，发还给主机。为了降低数据从内存中写入固态硬盘的延迟，固态硬盘会内置一块内存缓冲区，用于临时存放要写入闪存的数据。数据从主机传入固态硬盘时，会先临时存储在内存缓冲区中，之后异步地写入闪存介质中。

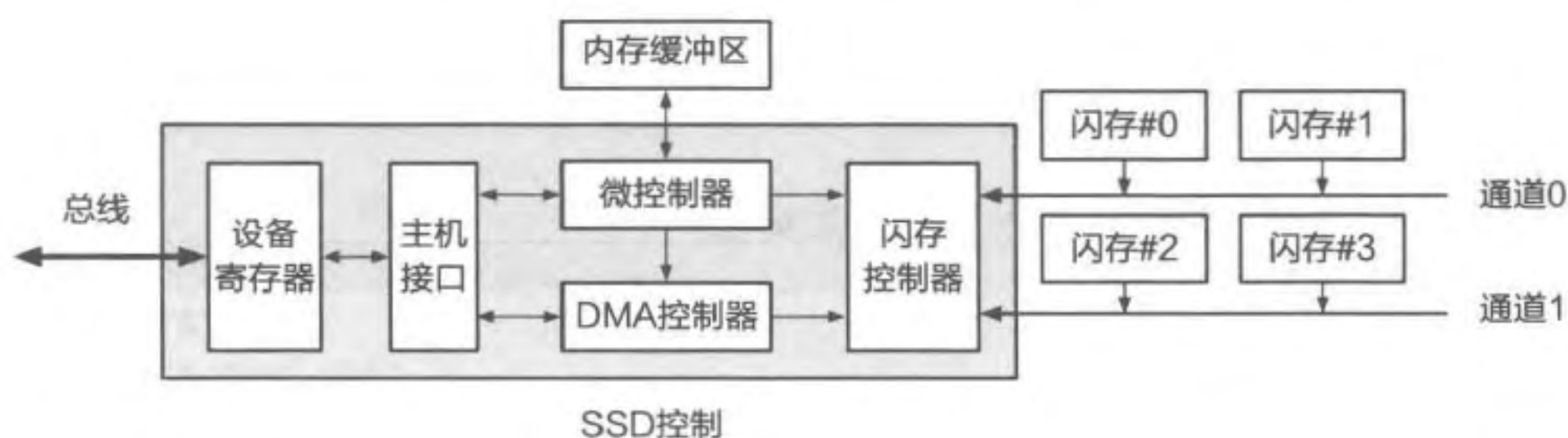


图 13.9 固态硬盘的结构

垃圾回收。由于无法直接覆盖闪存页中的数据，被修改的数据在闪存存储中会有多个版本。为了回收旧的数据，固态硬盘控制器通常会记录每个页是否有效。当一个块中的所有页都变为无效时，控制器将对这个块进行擦除操作；在擦除操作完成之后，控制器将这个块标记为可用，后续的写入或修改操作可以继续在该块的闪存页中进行。

并行性。固态硬盘没有机械硬盘磁头的机械限制，因此具有更高的并行性。固态硬盘中包含多个闪存存储单元，并提供多个访问通道（Channel）。如在图 13.9 中，有两个通道用于访问四个闪存存储。每个通道可以独立使用，因此固态硬盘可以同时处理多个存储请求。

闪存翻译层。固态硬盘中的存储介质以页为单位进行寻址，每个页有各自的物理地址（Physical Block Address）^①。在固态硬盘中存在两种地址：逻辑地址（LBA）和物理地址（PBA）。逻辑地址用于对固态硬盘外部呈现一个连续的地址空间，而物理地址则为内部闪存存储的实际地址。闪存翻译层（Flash Translation Layer, FTL）用于维护两套地址空间之间的映射。

机械硬盘的寻道和闪存的擦除均是比较慢的操作。机械硬盘的寻道往往出现在关键路径中，因此对性能有关键性的影响。而闪存的擦除却较少出现在关键路径中。通过闪存翻译层，新的数据可以写在已经被擦除的块中，而旧的数据可以被异步擦除。通常只有当缓存中剩余空间较少，同时缺少足量的被擦除但未被使用的块时，才会在关键路径上发生垃圾回收。等待垃圾回收往往会造成固态硬盘的读写性能下降。

13.2 设备发现与交互

本节主要知识点

- 操作系统是如何发现设备的？
- CPU 可以通过哪些方式访问设备寄存器？

① 为了与机械硬盘的称呼保持一致，固态硬盘虽然以页为单位进行寻址，但地址被称为块地址。

- ❑ 中断可以带来什么好处？
- ❑ 直接内存访问有哪些优点？
- ❑ 如何保证直接内存访问的安全性和正确性？

13.2.1 CPU 与设备的交互方式概览

尽管设备种类五花八门，但硬件制造商都约定俗成地采用“设备寄存器”作为 CPU 操作设备的方式。事实上，设备寄存器只是 CPU 与设备进行交互的基本方式之一，关于 CPU 如何与设备交互的更多细节将在本节展开介绍。

问题一：CPU 是如何找到设备的？设备种类繁多，CPU 需要一种方法去发现这些设备，这样才能进一步确认设备型号，从而选择对应的设备驱动与之交互。因此，计算机平台应提供一套设备发现机制，通知 CPU 当前平台已经成功连接的设备，并且告诉 CPU 设备寄存器所在的位置。

问题二：CPU 如何访问设备寄存器？CPU 在访问物理内存时，可以使用内存的物理地址；在访问设备时，该使用什么地址呢？硬件制造商也为设备设计了可寻址的地址空间，从而允许 CPU 通过访存指令，以访问内存的形式来读写设备寄存器（如图 13.10 中的 MMIO）。

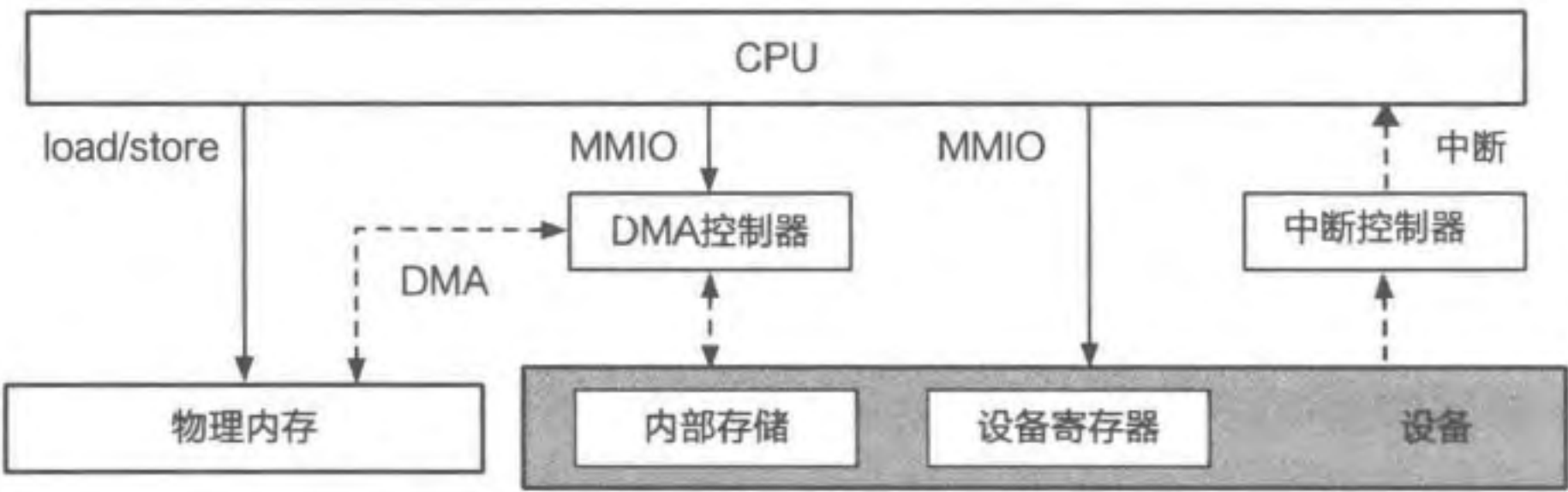


图 13.10 CPU 和设备的交互：既可以由 CPU 主动发起（向下箭头），也可以由设备主动发起（向上箭头）

问题三：CPU 还能如何与设备交互？除了可以让 CPU 通过寄存器访问设备外，硬件设备的制造商还提供了“直接内存访问”的方式（图 13.10 中的 DMA），允许设备和内存之间直接传输数据，不需要 CPU 参与。当设备完成数据传输后，会通过“中断”机制主动通知 CPU（图 13.10 中的中断）。

案例分析：e1000 网卡驱动的初始化

本节将围绕 e1000 网卡，介绍 CPU 和设备交互的硬件机制，以及驱动程序的基本组成。设备驱动主体可以分为配置信息和基本操作两大类。配置信息包括设备的寄存器空间、中断号、DMA 等信息，而基本操作则包括设备的初始化、工作模式切换，以及如何

同操作系统、应用程序进行交互等。代码片段 13.2 展示了 e1000 网卡驱动的配置信息和基本操作，以及网卡驱动的初始化流程 e1000_init()。

代码片段 13.2 e1000 网卡驱动的初始化伪代码

```

1 typedef struct e1000_dev_t {
2     // 网卡配置信息
3     struct net_info {
4         uint64_t io_base;           // MMIO 地址
5         uint64_t irq_num;          // 中断号
6         void * rx_base;             // DMA Rx 缓冲区
7         void * tx_base;             // DMA Tx 缓冲区
8         uint8_t mac_addr[6];       // 网卡 MAC 地址
9     } info;
10    // 网卡基本操作
11    struct net_ops {
12        int * transmit(void);       // Tx 发送函数
13        int * poll(void);           // Rx 轮询函数
14        int * control(void);        // 设备控制函数
15    } ops;
16 } e1000;
17
18 int e1000_irq_handler(void);       // e1000 中断接收函数
19 int e1000_poll(void);              // e1000 轮询接收函数
20 int e1000_transmit(void);          // e1000 发送函数
21 int e1000_control(void);           // e1000 控制函数
22
23 void e1000_init(void)
24 {
25     e1000 nic;
26
27     // 通过 PCI 总线探测 e1000 网卡是否存在
28     pci_probe(&nic, E1000_VENDOR_ID, E1000_PRODUCT_ID);
29     // 配置 MMIO 空间
30     ioremap(&nic.info.io_base);
31     // 读取网卡 MAC 地址
32     e1000_read_mac_addr(&nic.info.mac_addr);
33     // 配置 DMA 缓冲区
34     e1000_setup_rx_ring(&nic.info.rx_base);
35     e1000_setup_tx_ring(&nic.info.tx_base);
36     // 申请中断号，注册网卡中断，并使能中断
37     int irq_num = request_interrupt();
38     register_interrupt(&irq_num, e1000_irq_handler);
39     e1000_enable_irq(&nic.info.irq_num);
40
41     // 注册和网卡相关的操作
42     nic.ops.transmit = e1000_transmit;
43     nic.ops.poll = e1000_poll;
44     nic.ops.control = e1000_control;
45
46     // 将该网卡设备注册给操作系统

```



```
47  register_device(&nic, NET_DEVICE);
48 }
```

驱动初始化过程中，首先需要检查设备是否已经连接到计算机平台中，即设备发现（13.2.2 节）。e1000 是基于 PCI 总线的网卡设备，因此 e1000 网卡驱动通过 PCI 枚举流程索引到 e1000 网卡设备。驱动可以使用内存映射 I/O（Memory-Mapped I/O, MMIO）读写设备寄存器（13.2.3 节）。为了使用 MMIO，驱动在初始化时通过操作系统提供的 `ioremap` 接口，将网卡寄存器映射到驱动可以访问的地址空间中。借助 MMIO，网卡驱动读取网卡的 MAC 地址，同时配置用于收发数据的 Rx 和 Tx 队列。网卡数据收发使用直接内存访问（Direct Memory Access, DMA）的硬件机制（13.2.5 节）。最后，驱动向操作系统申请中断号，并使用该中断号将接收过程（即 Rx）注册为中断（Interrupt）。借助中断，网卡可以在 DMA 接收过程结束后主动通知 CPU（13.2.4 节）。

除了初始化网卡的配置信息之外，驱动程序还需要注册和网卡相关的基本操作。代码片段 13.2 中，e1000 网卡驱动既能选择基于中断的接收方法 `e1000_irq_handler`，也能选择基于轮询（Polling）的接收方法 `e1000_poll`。中断和轮询机制的细节和优劣比较将在 13.2.4 节展开讨论。发送则只采用 `e1000_transmit` 一种方法。网卡驱动还注册了控制函数 `e1000_control`，允许操作系统根据网络处理的实际情况，主动控制网卡的工作模式，比如在中断和轮询之间、在百兆和千兆的通信带宽之间按需切换等。最后，当配置信息和基本操作都初始化完成后，网卡驱动需要将自己注册到操作系统中，从而使整个系统的应用程序都能使用网卡进行通信。

13.2.2 设备发现

设备可以在操作系统启动之前就和 CPU 连接到同一主板平台上，也可以在操作系统运行时动态接入。为了帮助操作系统识别平台上连接好的设备，计算机系统需要提供一套相关的设备发现机制，帮助操作系统找到设备对应的寄存器区间。

如图 13.11 所示，在启动阶段，操作系统可以借助“设备树”或固件接口找到已经连接到平台上的设备，本质是查询一张现成的设备信息表。在运行阶段，动态接入的设备也能通过“热插拔”的方法进行识别，一般是依赖总线提供的发现机制。

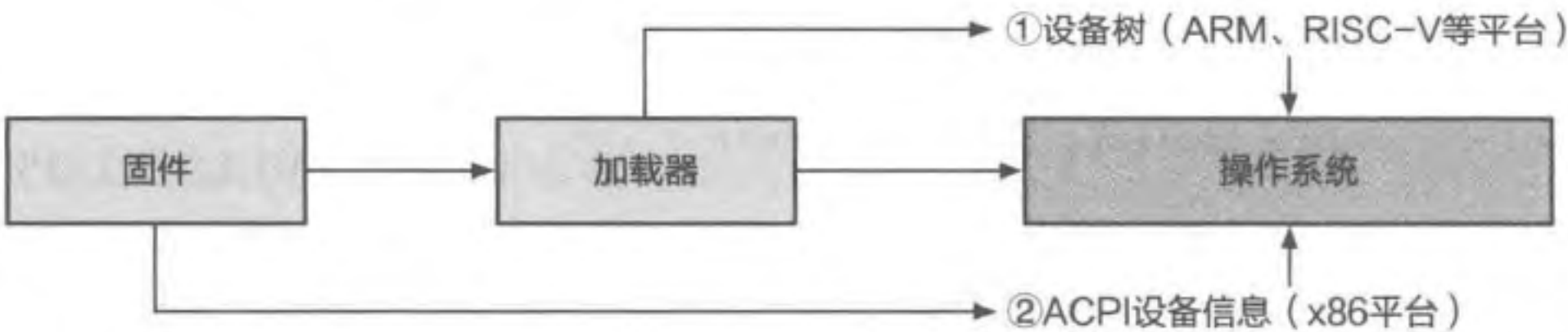


图 13.11 操作系统在启动阶段可以通过加载器传入的设备树获知平台的设备信息，或者通过固件接口查询设备的信息列表

启动阶段的设备发现

ARM 架构的系统常采用 SoC (System on Chip, 片上系统) 的设计, 将 CPU、内存和设备集成在单块芯片上。为了识别这些集成设备, 一种方法是直接将片上系统的设备信息硬编码到操作系统中, 从而在启动阶段直接找到对应的设备驱动对设备进行初始化和使能。然而, 片上系统的种类非常多, 同时相同平台的新版本也在陆续推出 (如本书实验所用的树莓派开发板就有多款型号), 因此将这些配置信息逐一编码到操作系统中会产生不小的维护成本。

在 ARM 架构上, 固件通常在系统启动之初, 在将控制权交给加载器 (Bootloader) 和操作系统之前, 负责对外设进行早期的初始化工作。开放固件 (Open Firmware) 标准定义了固件和操作系统之间的设备识别标准, 即设备树^[1]。设备树 (Device Tree) 是专用于描述硬件信息的数据结构, 具有可读性强的源码格式 (Device Tree Source, DTS) 和二进制格式 (Device Tree Blob, DTB) 两种。之所以称为“树”, 是因为设备树使用树状结构对 CPU、内存、总线、设备等硬件信息进行了层级化的组织。

以 ARM 平台的 ChCore 为例, 在启动阶段由引导程序 (如 U-Boot) 负责同时将 ChCore 系统镜像和设备树二进制文件 (DTB) 加载到物理内存中, 同时引导程序将 DTB 的地址传给 ChCore 系统。设备树中的 `compatible` 属性直接标注了当前片上系统的类型、版本等具体信息。ChCore 在启动阶段对 DTB 文件进行解析, 获取设备树中标注的具体设备信息, 匹配合适的驱动程序代码, 同时找到设备对应的寄存器区间。ARM 平台的其他系统 (包括 Linux) 也采用相同的设备发现流程。

代码片段 13.3 展示了树莓派 B+ 开发板对应的 BCM2835 片上系统的设备树源码, 可以看到 CPU、内存以及 SoC 上的所有设备都挂载在根目录 “/” 下, 而系统定时器、串口等设备都挂载在 `soc` 目录下。设备树源码直接标注了设备寄存器组在总线地址空间上的基地址和区间长度, 如系统定时器的基地址是 `0x7E003000`, 长度是 `0x1000`, 而 0 号串口 UART 的基地址是 `0x7E201000`, 长度是 `0x200`。操作系统在访问设备寄存器时, 不能直接使用总线地址, 而是需要将地址转换为对应的物理地址或虚拟地址。具体方法我们将在下一节中介绍。

代码片段 13.3 BCM2835 片上系统的设备树源码 (节选)

```
1 {
2     compatible = "raspberrypi,model-b-plus", "brcm,bcm2835";
3     model = "Raspberry Pi Model B+";
4
5     cpus {
6         cpu@0 {
7             device_type = "cpu";
8             compatible = "arm,arm1176jzf-s";
9             reg = <0x0>;
10        };
11    };
```



```

12
13 memory@0 {
14     device_type = "memory";
15     reg = <0 0x20000000>;
16 };
17
18 soc {
19     ranges = <0x7e000000 0x20000000 0x02000000>;
20     compatible = "simple-bus";
21
22     system_timer: timer@7e003000 {
23         compatible = "brcm,bcm2835-system-timer";
24         reg = <0x7e003000 0x1000>;
25     };
26
27     uart0: serial@7e201000 {
28         compatible = "arm,pl011", "arm,primecell";
29         reg = <0x7e201000 0x200>;
30     };
31     ...
32 };
33 };

```

并非平台上所有的硬件设备信息都需要记录在设备树中。设备树的设计初衷是记录平台上无法发现的设备信息。由于使用 USB、PCI 等总线的设备可以借助总线直接被操作系统发现，因此设备树无须对这部分信息进行记录，利用总线自身的协议即可完成对设备的识别。

小知识：基于 ACPI 的设备发现

x86 平台的计算机在启动阶段采用了另一套方案——基于高级配置与电源接口（ACPI）的设备发现^[2]。操作系统可以通过固件提供的 ACPI 接口获取硬件设备的信息列表。ACPI 接口还支持对设备进行电源管理的操作。考虑到 ACPI 在服务器领域的广泛使用，ARMv8 服务器也支持 ACPI[⊖]，原因之一是微软的 Windows 系统并不支持设备树。为了在 ARM 服务器上运行 Windows 系统，ARMv8 服务器提供对 ACPI 的支持。ARM 平台的 Linux 系统则同时支持设备树和 ACPI 两套设备识别方案。

运行阶段的设备发现

计算机系统某些设备会在运行阶段动态接入或断开，也叫作设备的热插拔。通常来说，运行阶段的设备发现需要有总线的支持。支持热插拔的总线可以感知新设备的接入，并将设备接入事件通知给 CPU，CPU 上的操作系统进而调用总线驱动去获取设备

⊖ 参见 <https://www.kernel.org/doc/html/latest/arm64/arm-acpi.html>。

的具体类型。为了使能新接入的设备，操作系统需要为新设备分配对应的寄存器空间。举例而言，当新接入一个即插即用（Plug and Play）的 PCI 设备时，PCI 总线会将新设备的接入事件通知给操作系统。操作系统通过读取 PCI 总线的 B/D/F 获取设备的有关信息，并为接入的设备配置设备寄存器对应的区间。操作系统的设备驱动通过这块区间读写设备寄存器，进而驱动该 PCI 设备。

13.2.3 设备寄存器的访问

设备寄存器位于设备控制器内部而不在 CPU 内部，那么 CPU 是如何得到这些设备寄存器的地址的呢？在现代计算机平台上，硬件设计者往往在物理地址空间上给 I/O 设备分配了一段专门的地址区域，用于映射设备寄存器。由于该段地址已经被设备占据，因此不能再用于物理内存。以本书配套使用的树莓派开发板为例，图 13.12 展示了树莓派开发板对应的 BCM283x 系列 SoC 的物理地址映射，其中物理地址的 0x20000000 到 0x20FFFFFFF 专门用于映射设备寄存器。将物理内存和设备寄存器进行统一编址的方式，允许运行在 CPU 上的操作系统通过访存指令（如 ARM 的 `ldr/str` 指令）直接操作设备寄存器，进而获取设备状态并对设备进行配置。这种将设备寄存器直接映射到物理地址空间中的设备操作方式称为内存映射 I/O（Memory-Mapped I/O，MMIO）。

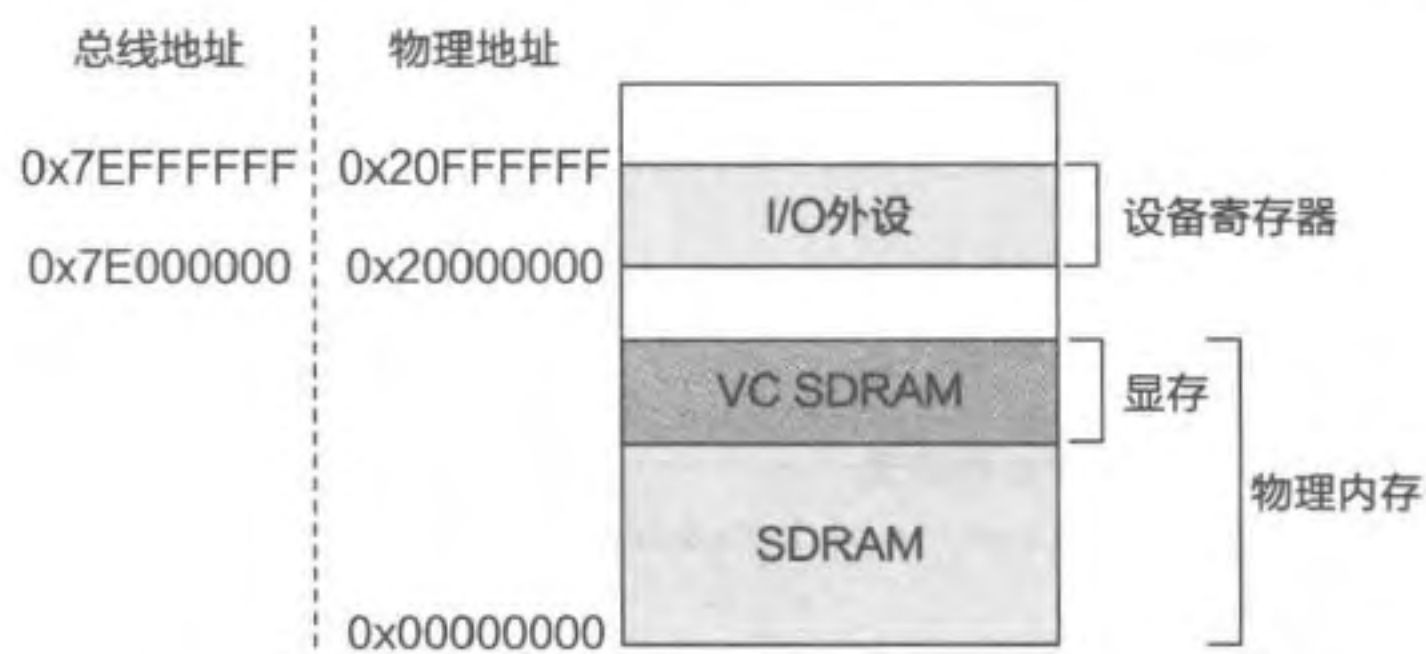


图 13.12 BCM283x 系列 SoC 的物理地址映射：物理内存和设备寄存器占用不同的物理地址

小知识：MMIO 的编程注意事项

在 MMIO 方式下，用操作内存的方法直接操作设备寄存器并不能保证每次都成功操作设备。因为无论是编译器还是 CPU 都会对内存读写指令进行优化，导致对设备访问结果的不正确。

- 编译器会对内存的读写指令生成进行优化。例如，当程序连续读取同一地址时，由于没有写操作，因此编译器可以将多次读合并为一次读，或只读取 CPU 寄存器而不读取内存。但对设备寄存器的读而言，连续两次读可能读到设备更新后的最新数据或状态，也就是不同结果。为了避免编译器的优化，通常对于 MMIO 的地址区间，应该在程序中使用 `volatile` 关键字。

- 现代 CPU 采取各种技巧来提升性能，如内存操作的乱序执行、分支预测以及各种缓存机制。为了避免 CPU 乱序执行导致设备寄存器的操作结果不正确，程序还需要在 MMIO 访问时插入内存屏障（Memory Barrier），保证设备寄存器操作的顺序执行。

设备寄存器也可以采用独立于物理地址空间之外的特殊地址空间进行寻址，如 x86 平台的 I/O 端口空间。相应的设备操作方式称为端口映射 I/O（Port-Mapped I/O, PMIO）。在 PMIO 中，CPU 需要使用特殊的 I/O 指令对设备寄存器进行读写，比如 x86 平台的 PMIO 提供了相应的 in/out 指令。

内存映射 I/O 和端口映射 I/O 被统称为可编程 I/O（Programmed I/O, PIO），因为它们允许驱动的开发根据根据需要编程设备的具体行为。代码片段 13.4 展示了用于读写设备寄存器的 WRITE_REG 宏和 READ_REG 宏的 MMIO 及 PMIO 实现。其中 MMIO 实现和 C 语言中的指针用法一样，但加入了 volatile 关键字和内存屏障，以防止编译器和 CPU 的错误优化；而 PMIO 则使用专门的 inl/outl 指令操作 32 位字长的 I/O 端口。

代码片段 13.4 内存映射 I/O 与端口映射 I/O 的接口和实现

```

1 // 驱动代码中读写设备寄存器的专用宏
2 #define WRITE_REG(a, reg, value) \
3     writel((value), ((a)->hw_addr + reg))
4
5 #define READ_REG(a, reg) \
6     readl((a)->hw_addr + reg)
7
8 // ARM、x86 上基于内存映射 I/O 的实现
9 void writel(uint32_t value, void *addr)
10 {
11     wmb();          // 写内存屏障
12     *(volatile uint32_t *)addr = value;
13 }
14 unsigned int readl(void *addr)
15 {
16     return *(const volatile uint32_t *)addr;
17     rmb();          // 读内存屏障
18 }
19
20 // x86 上基于端口映射 I/O 的实现
21 void writel(uint32_t value, unsigned port)
22 {
23     asm volatile("outl %0, %w1" : : "a"(value), "Nd"(port));
24 }
25 uint32_t readl(unsigned port)
26 {
27     uint32_t value;

```

```
28- asm volatile("inl %w1, %0" : "=a"(value) : "Nd"(port));
29  return value;
30 }
```

案例分析：通过 e1000 的设备寄存器访问网卡内部存储

EEPROM 是位于网卡设备内部的闪存存储介质。为了帮助 CPU 读取存储在 EEPROM 上的数据，网卡为软件暴露了 EERD (EEPROM Read Register)。EERD 是一个 32 位长的设备寄存器，不同位的含义如表 13.2 所示。

表 13.2 e1000 网卡上的 EERD 设备寄存器

域	位	含 义
START	0	提醒网卡开始读取 ADDR 位的数据
DONE	1	当值为 1 时，表示数据读取结束
ADDR	15 : 2	EEPROM 上待读取数据的地址
DATA	31 : 16	EEPROM 返回的数据内容

代码片段 13.5 展示了通过读写 e1000 网卡上的 EERD 设备寄存器，获取 EEPROM 存储介质上的信息的过程。e1000 只支持一次从 EEPROM 中读取一个字 (Word) 的长度，即 2 字节的数据。驱动程序首先需要将待读取数据的地址写入 EERD 寄存器的 ADDR 域，同时将 EERD 寄存器的开始位标记为 1，提醒网卡控制器地址信息已经配置完毕。网卡控制器收到地址信息后，会去寻址 EEPROM 中相应地址的数据，并将读到的数据写入 EERD 寄存器的 DATA 域。CPU 上的软件需要不断查询 EERD 寄存器的 DONE 域，不停判断网卡控制器是否完成了数据的读取操作。一旦 DONE 域显示数据读取完成，驱动便可以读到从网卡返回的 EEPROM 中一个字的数据。

代码片段 13.5 通过 READ_REG 和 WRITE_REG 操作 e1000 网卡设备上的 EERD 设备寄存器，获取存储在 EEPROM 上的数据

```
1 #define EERD 0x00014
2 #define EERD_START 0x00000001
3 #define EERD_DONE 0x00000010
4 #define EERD_ADDR_SHIFT 2
5 #define EERD_DATA_SHIFT 16
6
7 int32_t e1000_poll_eerd_done(struct e1000_hw *hw);
8
9 int32_t e1000_read_eeprom(struct e1000_hw *hw,
10     uint16_t offset, uint16_t words, uint16_t *data)
11 {
12     uint32_t i, eerd = 0;
13     int32_t error = 0;
14
15     for (i = 0; i < words; i++) {
16         // 将预读取的数据地址写入地址位，同时将 START 置为 1
17         eerd = ((offset+i) << EERD_ADDR_SHIFT) + EERD_START;
18         WRITE_REG(hw, EERD, eerd);
19
20         // 查询网络接口控制器是否完成操作 (见代码片段 13.3)
```

```

21     error = e1000_poll_eerd_done(hw);
22     if (error) break;
23
24     // 从数据位读取网卡返回的数据
25     data[i] = (READ_REG(hw, EERD) >> EERD_DATA_SHIFT);
26 }
27 return error;
28 }

```

13.2.4 中断

虽然 CPU 和设备之间有总线连接，但是 CPU 并不清楚设备当前所处的真实工作状态。设备可能处于“就绪”或“忙碌”的状态，在忙碌阶段就无法响应 CPU 的请求。CPU 必须通过设备寄存器查询设备的工作进度，一旦设备就绪，就可以立刻发出新的操作命令。

在代码片段 13.6 展示的 e1000 网卡驱动代码示例中，CPU 需要以不断循环等待的方式轮询（Polling）EERD 寄存器的 DONE 域，才能得知网卡控制器是否已经完成 EEPROM 的读请求。类似地，网络数据包的接收请求也可以采取轮询的方式，由 CPU 主动检查网卡当前收取数据的情况。然而，来自外部的网络数据包究竟何时能到达是非常难以确定的，对于高速运行的 CPU[⊖]而言，轮询期间网卡代码始终占用 CPU 资源，并没有执行有意义的工作。

代码片段 13.6 轮询 10 万次查询 EEPROM 数据读取完成情况

```

1 #define EERD 0x00014
2 #define EERD_DONE 0x00000010
3
4 int32_t e1000_poll_eerd_done(struct e1000_hw *hw)
5 {
6     uint32_t attempts = 100000;
7     uint32_t i, reg = 0;
8
9     for (i = 0; i < attempts; i++) {
10         reg = READ_REG(hw, EERD);
11         if (reg & EERD_DONE) return SUCCESS;
12         udelay(5);
13     }
14     return ERROR;
15 }

```

更为合理的做法是避免 CPU 总是处于轮询的工作方式，而是让网卡只有在网络包到来时才主动通知 CPU。这种由硬件设备在某些事件（如网络包到来）发生后再通知 CPU

⊖ 现在的处理器可达到 GHz 的处理频率，即 1 纳秒可以执行若干条指令。

的机制被称为中断（Interrupt）。中断机制使得设备可以向 CPU 主动发送中断信号：CPU 在还没收到中断时可以处理其他任务；CPU 收到中断后，正在 CPU 上执行的任务会被打断，而相应的中断处理函数（Interrupt Handler 或者 Interrupt Service Routine, ISR）会被执行并立刻响应中断事件。不同于系统调用和同步异常（比如内存访问指令导致的缺页），设备中断是异步发生的，即 CPU 的执行过程可能随时被设备发送的中断打断。图 13.13 展示了 CPU 的中断响应流程。

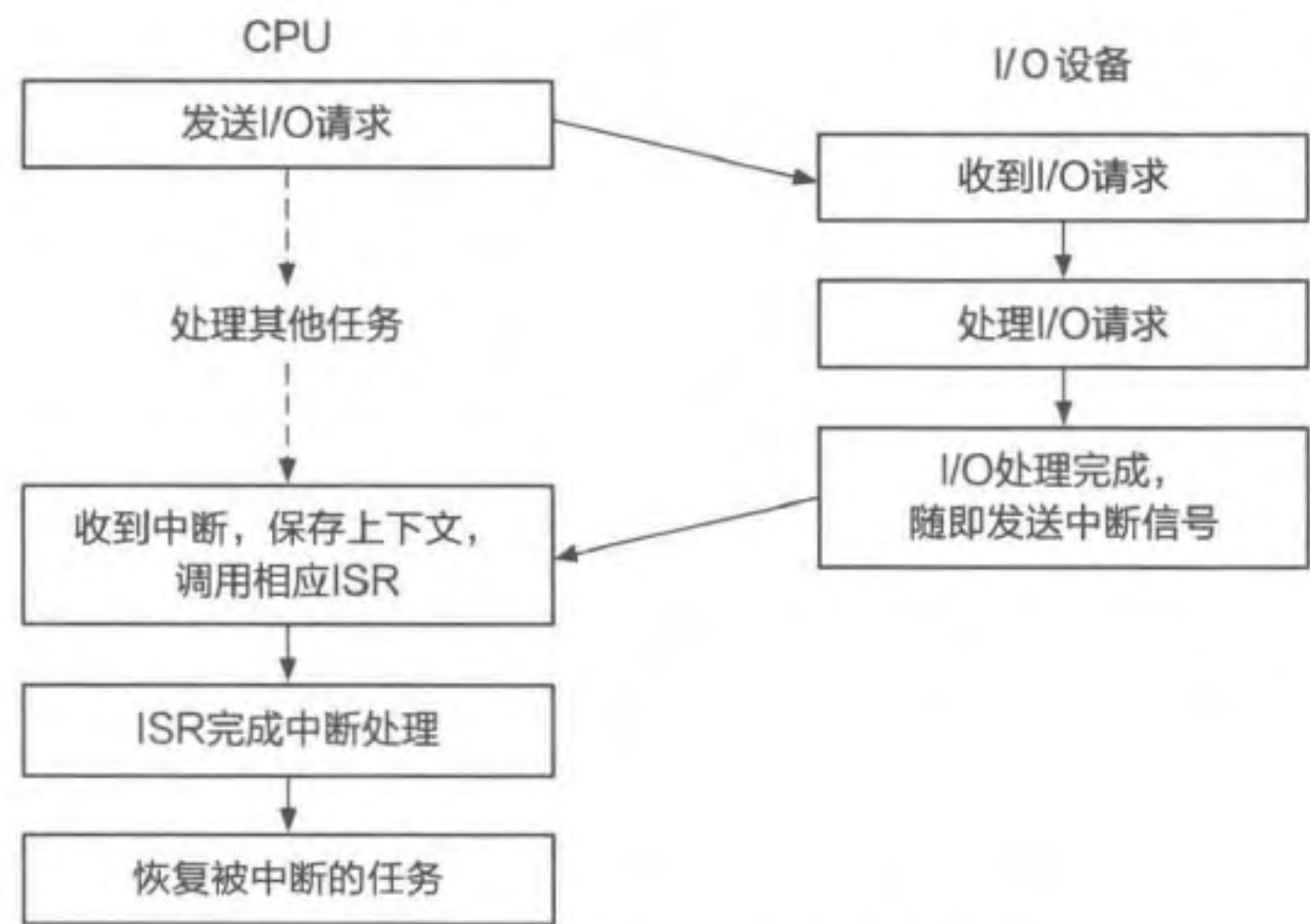


图 13.13 CPU 的中断响应流程

如果设备需要完成的操作相对耗时，采用中断的方式能够有效节省 CPU 资源。比如，当操作系统需要读取硬盘中存储的数据时，可以在向硬盘发出读取请求后直接进行调度，继而让 CPU 运行某个可执行的进程，而不是轮询等待硬盘完成读取操作；硬盘完成操作后再向 CPU 发出中断，在中断处理函数中，操作系统可以知道之前的硬盘读取操作已经完成。

中断的异步特性并非对所有的设备都有节省 CPU 资源的好处。例如，对于串口设备而言，由于发送字符的操作所需要的执行时间很短，如果也利用中断机制实现成异步的方式，不仅对于提高 CPU 资源的利用率而言毫无益处，甚至可能因为中断的引入而导致过多的 CPU 资源占用（因为中断的上下文切换也需要花费 CPU 时间）。

由于频繁中断会导致 CPU 产生大量上下文切换，部分设备（如高性能网卡）提供了将多个中断合并为单个中断的能力，即中断合并（Interrupt Coalescing）。中断合并利用了设备内部的 FIFO 缓冲区，设备收到数据后，可以不马上通知 CPU，而是将数据缓存在设备内部的 FIFO 上，当数据个数超过一定阈值或者超过预先配置的超时时间时，设备会立刻触发中断。理想情况下，中断合并可以将中断的触发频率显著降低，代价是中断的响应延迟会有所增加。

对于设备的某个操作，驱动采用轮询还是中断的方式并非一成不变的。以网卡设备

接收网络包的操作为例，由于什么时候收到网络包是不确定的，采用轮询的方式可能会导致大量 CPU 资源的浪费，因此采用中断方式响应网络包的到达可能是默认情况下更好的选择。但是，当网卡设备的性能足够好，网络包的到来连续不断甚至足够高频时，网卡驱动采用轮询的方式能够避免频繁地触发中断引起的上下文切换，反而能够更加高效地对网络包进行处理。在 Linux 操作系统中，网卡驱动首先采用中断方式接收网络包，当操作系统发现网络包的到达速率达到一定阈值后，将改为轮询方式；而当一段时间不再收到网络包或收包速度显著变慢后，再切换回中断方式。该方法称为 New API，简称 NAPI^[3]。

案例分析：e1000 的 Rx 中断分析

如代码片段 13.7 所示，对 e1000 网卡进行初始化时，网卡驱动需要向操作系统注册对应的中断处理函数。这样，当 e1000 网卡的接收队列（即 Rx 队列）收到来自其他计算机的一个完整数据包时，网卡会主动向 CPU 发送中断。CPU 收到中断后会跳转到注册好的中断处理函数 `e1000_irq_handler`。`e1000_irq_handler` 首先修改网卡的 IMC（Interrupt Mask Clear）设备寄存器，屏蔽后续的网络卡中断，以保证网络收包处理过程的原子性。随后检查网卡 Rx 队列中该包所在描述符的状态位，确认网络包当前处于就绪状态，即数据已经成功到达。确认成功后，网卡驱动从 Rx 队列中取走该网络包，并交给操作系统的相关函数进行处理（通常是操作系统的网络协议栈）。操作系统处理完成后，中断处理函数会使能网卡中断，以允许下一次的网络中断到来。IMS（Interrupt Mask Set/Read）用于使能 e1000 网卡的中断能力。

代码片段 13.7 e1000 的中断处理

```

1 #define IMS_ENABLE_MASK 0x9D
2 #define E1000_RXD_STAT_DD 0x01
3
4 static inline void enable_irq(struct e1000_hw *hw)
5 {
6     WRITE_REG(hw, IMS, IMS_ENABLE_MASK);
7 }
8
9 static inline void disable_irq(struct e1000_hw *hw)
10 {
11     WRITE_REG(hw, IMC, ~0);
12 }
13
14 int e1000_irq_handler(struct e1000_hw *hw)
15 {
16     // 屏蔽网卡中断，保证网卡中断处理的原子性
17     disable_irq(hw);
18
19     // 检查网卡的接收队列是否处于就绪状态
20     struct e1000_rx_desc *rd = rx_base;
21     if (!(rd->status & E1000_RXD_STAT_DD))
22         return ERROR;

```

```

23
24 // 将网络包交给操作系统进行处理
25 net_rx((char *)rd->addr, rd->length);
26
27 // 使能网卡中断, 使用默认的掩码配置
28 enable_irq(hw);
29 return SUCCESS;
30 }
31
32 void e1000_init(struct e1000_hw *hw)
33 {
34 // 向操作系统申请中断号, 注册中断处理函数
35 int irq;
36 register_interrupt(&irq, e1000_irq_handler);
37 enable_irq(hw);
38 }

```

中断控制器

尽管硬件机制提供了设备向 CPU 主动发送中断的能力, 然而在多设备场景下, CPU 并不知道中断对应的是哪一个设备, 在多核 CPU 场景下, 中断发生时也不知道该发给哪一个核心进行处理。为了提供中断的可编程能力, 硬件设计者在 CPU 与设备之间引入了中断控制器 (Interrupt Controller), 专门负责中断的配置和管理。中断控制器通常提供如下几方面能力:

- **中断号**: 用于映射设备到中断的关系。中断发生时, 操作系统可以根据中断号快速判断中断来自哪个设备, 并采取预定义的方式快速响应中断。
- **中断屏蔽**: 用于保证 CPU 上某些操作的原子性。例如, 为了保证临界区操作的原子性, 可以通过屏蔽时钟中断的方法, 防止当前任务被调度走。
- **中断优先级**: 用于区分更为重要或紧迫的中断。例如, 当温度过高时, 温度监控设备就应及时向 CPU 报告, CPU 需要立刻调高风扇转速, 以帮助系统快速降温。此时, 温度监控设备的中断就会比其他设备的中断优先级更高。

小知识: ARM 架构的通用中断控制器

ARM 架构的中断控制器称为通用中断控制器 (Generic Interrupt Controller, GIC)^[4]。

如图 13.14 所示, GIC 有两个组成模块:

- **分发器 (Distributor)**: 多核场景下, 分发器负责汇聚所有设备的中断请求, 并根据路由配置将中断分发给目标核心。分发器用于提供全局的中断使能、中断优先级配置等能力。
- **CPU 接口 (CPU Interface)**: CPU 接口与 CPU 核心相连, 用于提供每个核心的本地中断管理能力, 如单个核心的中断使能、中断确认等。CPU 接口还可用于发起核间中断。

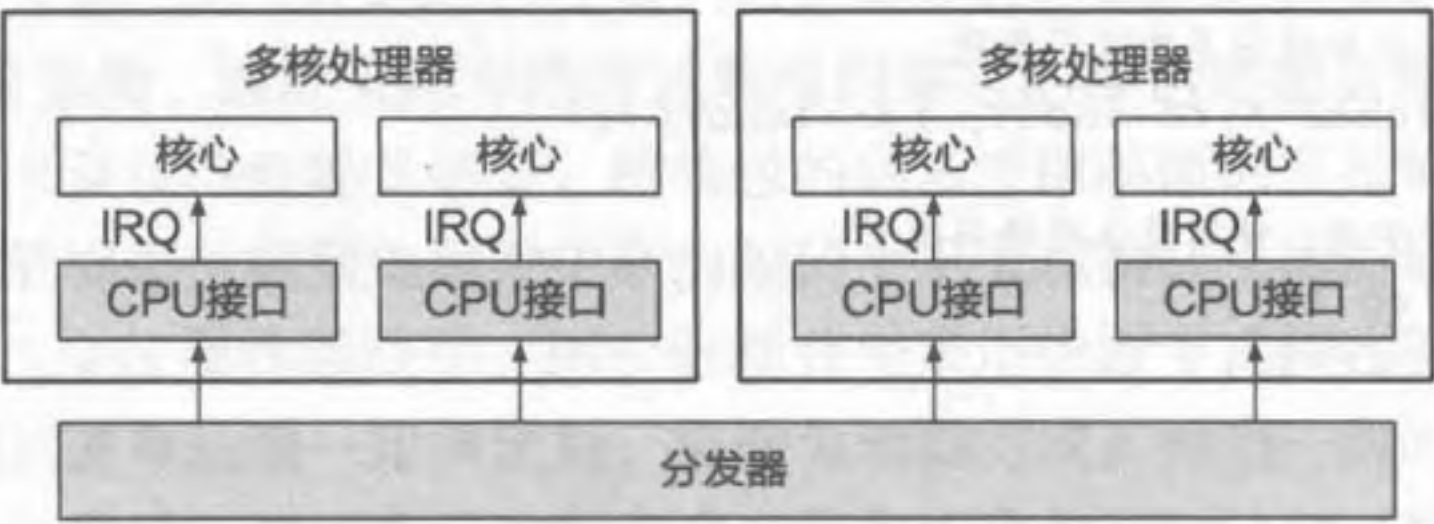


图 13.14 GIC 中断控制器分为两个模块，分发器负责全局的中断路由，CPU 接口负责每个 CPU 核心的中断管理

GIC 的中断响应流程。每个设备都通过总线互联的方式和 GIC 中断控制器连接。当设备的特定状态被满足后，设备会向 GIC 中断控制器发送中断请求。GIC 收到中断请求后，分发器会将优先级最高的中断派发给相应的 CPU 接口，进而由 CPU 接口转发给 CPU 核，执行 CPU 核上预先注册好的中断处理程序。中断处理程序确认中断时，会读取 CPU 接口上的寄存器，查询具体产生的中断号和中断类型。对于不同设备来源的中断，GIC 通过查表的方式得到不同中断处理的函数入口，这个表被称为**中断向量表（Interrupt Vector Table）**。

13.2.5 直接内存访问

对于操作系统和硬件设备之间的数据交换，上文介绍的读写设备的数据寄存器是一种方式，该方式对于小数据量的数据传输是合适的。但是，当需要传输的数据量较大时（比如大量网络数据包的收发），继续使用每字节读写寄存器的方式是不合适的，因为这会占用大量的 CPU 时间。为解决该问题，硬件提出了**直接内存访问（Direct Memory Access, DMA）**技术，允许硬件设备绕过 CPU 进行批量物理内存数据的读写。有了 DMA，在硬件设备传输数据的同时，CPU 可以执行其他任务，因此可以显著提高 CPU 的有效利用率。

CPU 和硬件设备都可以发起 DMA 操作，即把数据从物理内存发送给设备，或者把设备的数据拷贝到物理内存中。以 CPU 发起 DMA 为例，操作系统首先在内存中分配一块 DMA 缓冲区，随后发起 DMA 请求，设备在收到请求后通过 DMA 的方式将数据批量传输至 DMA 缓冲区。DMA 操作完成后，设备触发中断，通知操作系统对 DMA 缓冲区中的数据进行处理。

通常，DMA 过程需要 DMA 控制器的参与。DMA 控制器可以是单独的设备，与 CPU 和物理内存一同连接到可以进行数据交换的总线上。不过在现在的计算机系统中，支持 DMA 的设备内部一般带有 DMA 控制器。图 13.15 给出了在 DMA 控制器参与下的 CPU 发起 DMA 的具体流程：

1. CPU 向 DMA 控制器发送 DMA 缓冲区的位置和长度，以及数据传输的方向，随后放弃对总线的控制权（图 13.15a）。
2. DMA 控制器获得总线控制权（图 13.15b）。
3. DMA 控制器根据从 CPU 获得的指令，将设备的数据拷贝至内存，其间 CPU 可以执行其他任务（图 13.15c）。
4. DMA 控制器完成 DMA 后向 CPU 发送中断，通知 DMA 已经完成。此时 CPU 会重新得到总线的控制权（图 13.15d）。

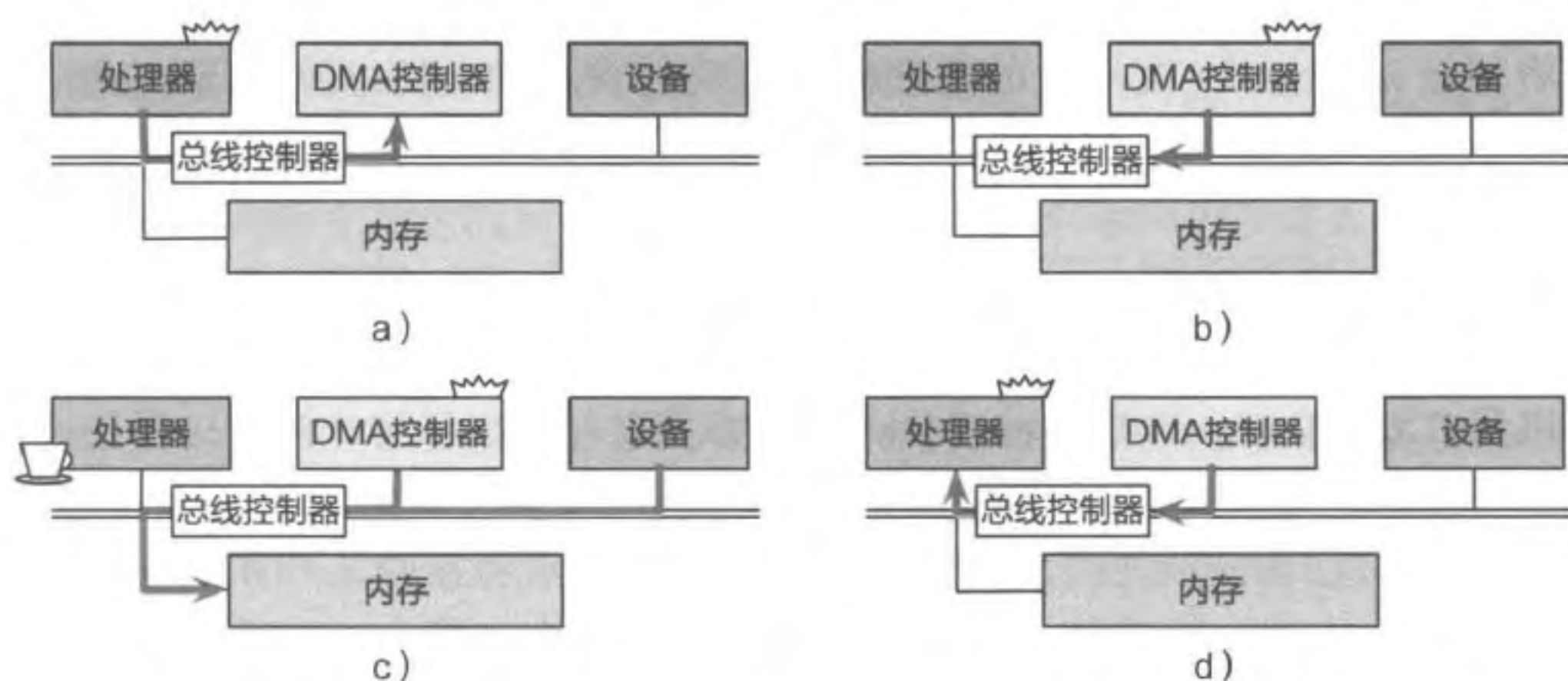


图 13.15 DMA 过程示意图。注意，实际上 DMA 控制器一般位于设备内部

值得一提的是，运行在 CPU 上的操作系统依然是通过 MMIO 的方式对（设备中的）DMA 控制器发起控制命令的。当然，设备也可以主动发起 DMA 操作，不过通常也需要操作系统为其预先分配好 DMA 缓冲区，否则设备并不知道 DMA 的目标内存地址。例如，操作系统首先为网卡设备分配物理内存作为网络包接收队列，并且通过 MMIO 的方式把分配好的物理内存告诉网卡，网卡设备在收到网络帧后，可以通过 DMA 的方式把数据写入物理内存中。本节的末尾我们将以 e1000 为例重点描述如何配置网卡以完成 DMA 操作。

小知识：SMMU

由于设备在进行 DMA 操作时能够直接访问物理内存，受到攻击的设备或者存在缺陷的设备可能会泄露或破坏任意内存数据。为此，ARM 架构提出 SMMU 机制（System MMU），避免设备直接使用物理地址。类似 MMU 将虚拟地址翻译到物理地址，SMMU 负责将硬件设备使用的 I/O 内存地址翻译到实际的物理地址。不过，SMMU 不是 ARM 架构中必须支持的机制，也并非所有的 ARM 平台都支持 SMMU。在 x86 架构中，SMMU 被称为 IOMMU。

现代处理器中通常有多级的高速缓存（CPU Cache），物理内存的数据并非始终是最新的，这就导致 DMA 过程中设备应该访问的正确数据还滞留在高速缓存中，没有被及时同步到物理内存中。操作系统该如何处理这种情况呢？

针对 DMA 发生时 DMA 缓冲区的数据和 CPU 高速缓存不一致的情况，通常有三种解决方法：

- 将用于设备 DMA 缓冲区的物理内存页映射为 Non-cacheable，即 CPU 对该 DMA 缓冲区的任何访问都绕过 CPU 高速缓存，从而避免出现数据不一致的情况。代价是 CPU 读写 DMA 缓冲区的性能不高。
- 将用于设备 DMA 缓冲区的物理内存页映射为 Cacheable，但是 CPU 对 DMA 缓冲区的每次更新操作都需要显式将内容从高速缓存同步到物理内存中。该做法对驱动开发人员的代码编写正确性有很高的要求。
- 由硬件在总线互联上直接提供保证 DMA 一致性的能力，如总线监听（Bus Snooping）技术。典型代表有 AMBA 总线标准的 ACE（AXI Coherency Extensions）^[5]，以及 CXL（Compute Express Link）^[6] 协议等。

案例分析：通过 e1000 的 DMA 接收网络数据

为了加速 CPU 和网卡之间的数据交换，网卡设计者将接收队列和发送队列分别设计为 DMA 队列——Rx 队列和 Tx 队列。在 DMA 队列中，每个以太网帧用一个描述符（Descriptor）表示。如代码片段 13.8 所示，描述符中包含一个以太网帧在物理内存的起始地址和长度，以及描述符的状态信息和错误信息等。不同厂商的网卡描述符的定义和个数不尽相同。

代码片段 13.8 e1000 网卡的 DMA Rx 描述符

```
1 struct e1000_rx_desc {
2     volatile uint64_t addr;
3     volatile uint16_t length;
4     volatile uint16_t checksum;
5     volatile uint8_t status;
6     volatile uint8_t errors;
7     volatile uint16_t special;
8 };
```

为了使 CPU 能收到网卡的数据包，网卡驱动需要对网卡的描述符进行配置。为了帮助读者理解后续驱动代码中设备寄存器的含义，我们将涉及的 e1000 寄存器信息列在表 13.3 中。代码片段 13.9 配置了 128 个 Rx 描述符。对于每个描述符，都分配一块物理连续的内存与之对应，从

表 13.3 e1000 网卡和 Rx 描述符队列相关的主要寄存器及其含义

目 的	寄存器	含 义
配置接收队列	RDBA	配置 Rx 描述符队列的基地址
	RDLEN	配置 Rx 描述符队列的总长度
	RDH	配置 Rx 描述符队列的头指针
	RDT	配置 Rx 描述符队列的尾指针

而允许网卡通过描述符中的地址找到内存中的数据。这里假定 `kzalloc` 函数返回的是物理地址，如果使用虚拟地址的话，还需要将其转换成物理地址才能使用，因为 DMA 控制器访问的是物理内存（假设没有 SMMU）。此外，初始化阶段还要把描述符的整体信息透露给网卡，包括描述符环形队列的头尾指针，这样网卡才能正常工作。

如图 13.16 所示，e1000 网卡使用首尾两个指针来管理 Rx 描述符队列，由此模拟出环形队列的效果。初始化时，头指针默认指向 0，而尾指针指向队列尾部，此时网卡可以接收数据。网卡收到一个完整的数据帧后，数据帧会以 DMA 的方式拷贝到描述符中指定的预分配好的物理内存中，同时由网卡更新头指针寄存器。图 13.16

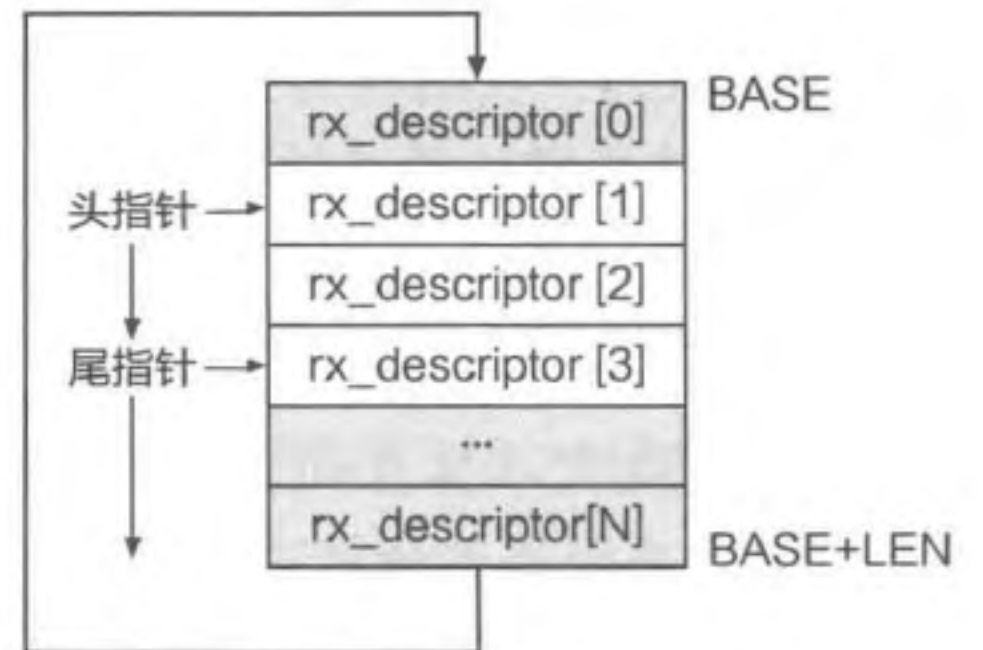


图 13.16 e1000 的 Rx 描述符环形队列

中的彩色部分表示网络帧已经被硬件 DMA 传输到内存中但尚未被网卡驱动接收的描述符。当头指针与尾指针相等时，表示环形队列被填满了完成 DMA 的数据帧，网卡不再接收新的数据帧，以防止发生覆盖。网卡驱动通过描述符中的 `status` 字段来确认数据帧是否已经就绪，就绪则意味着该数据帧可以被读取。当网卡驱动读走一个完整的数据帧后，需要推进对应的尾指针寄存器，告知网卡该数据帧已被读走。只有当尾指针推进时，网卡才能持续地通过 DMA 传输数据并发送中断以通知 CPU。

代码片段 13.9 e1000 网卡：初始化 DMA 接收描述符

```

1 #define RXBUF_COUNT 128
2 #define RXBUF_SIZE 2048
3
4 // e1000 接收队列的指针列表
5 struct e1000_rx_desc rx_base[RXBUF_COUNT];
6
7 void e1000_setup_rx_ring(struct e1000_hw *hw)
8 {
9     // 逐个初始化 Rx 描述符
10    for(int n = 0; n < RXBUF_COUNT; n++) {
11        rx_base[n]->addr = kzalloc(RXBUF_SIZE);
12        rx_base[n]->status = 0;
13    }
14
15    // 配置 DMA Rx 描述符环形队列的基地址和总长度
16    WRITE_REG(hw, RDBAL, (uint32_t)rx_base);
17    WRITE_REG(hw, RDBAH, (uint32_t)(rx_base >> 32));
18    WRITE_REG(hw, RDLEN, RXBUF_COUNT * sizeof(struct e1000_rx_desc));
19
20    // 初始化 DMA Rx 描述符环形队列的头指针和尾指针寄存器
21    WRITE_REG(hw, RDH, 0);

```



```
22 WRITE_REG(hw, RDT, E1000_NUM_RX_DESC - 1);
23 }
```

代码片段 13.10 将配置 DMA Rx 的逻辑加入了设备驱动的初始化流程，同时在 Rx 中断发生时从 DMA 描述符中获取数据帧的信息。在 Rx 中断发生时，Rx 中断处理函数会遍历 Rx 队列，找出所有就绪的数据帧，从描述符中获取对应的地址和长度，从而得到数据帧并传给操作系统的网络协议栈进行处理。

代码片段 13.10 基于 DMA 队列的 e1000 的中断处理

```
1 #define E1000_RXD_STAT_DD 0x01      // 描述符就绪
2 #define E1000_ICR_RXT0 0x80         // Rx 中断
3
4 int e1000_irq_handler(struct e1000_hw *hw)
5 {
6     ...
7     // 判断是否为 e1000 Rx 中断
8     unsigned irq = READ_REG(hw, E1000_ICR);
9     if (irq & E1000_ICR_RXT0) {
10         while (1) {
11             // 读取 Rx 尾指针，尾指针的下一个为可用的 Rx 数据帧
12             int rx_cur = READ_REG(hw, RDT);
13             rx_cur = (rx_cur + 1) % RXBUF_COUNT;
14
15             // 判断当前 Rx 缓冲区的数据是否就绪
16             if (!(rx_base[rx_cur]->status & E1000_RXD_STAT_DD))
17                 return ERROR;
18
19             // 将网络包交给操作系统的网络协议栈进行处理
20             uint64_t buf = rx_base[rx_cur]->addr;
21             uint16_t len = rx_base[rx_cur]->length;
22
23             net_rx((char *)buf, len);
24             rx_base[rx_cur]->status = 0;
25
26             WRITE_REG(hw, RDT, rx_cur);    // 更新 Rx 尾指针
27         }
28         return SUCCESS;
29     }
30     ...
31 }
32
33 void e1000_init(struct e1000_hw *hw)
34 {
35     int irq;
36     register_interrupt(&irq, e1000_irq_handler);
37     ...
38     e1000_setup_rx_ring(hw);
39 }
```

13.3 设备管理的共性功能

本节主要知识点

- 应用程序是如何找到设备的？
- 为什么要对设备进行分类？不同分类有什么特点？
- 为什么要设计不同的缓冲区？缓冲区是否是必要的？
- 为什么要提出 `ioctl` 系统调用？

尽管不同设备的功能各异，但从管理的角度来看，它们之间有着不少共同的需求。首先，操作系统需要为所有设备提供合适的抽象，以便应用程序对设备进行查找和访问。其次，操作系统需要根据设备功能对设备进行合理的分类，以便提取共性需求并复用功能相同的实现代码。再次，操作系统需要根据设备的特点提供合适的管理方案，比如根据设备的访问特性、交互速率选择合适的缓冲区方案。最后，考虑到设备的多样性，操作系统需要为应用程序提供合适的设备操作接口。

13.3.1 设备的文件抽象

为了帮助应用程序和设备进行交互（比方应用程序希望通过串口输出计算结果），操作系统需要为设备设计合理的软件抽象，同时分配合适的逻辑名称，以方便应用程序查找相应设备。既然操作系统的用户已经对文件这一抽象十分熟悉，那么能不能直接复用这个抽象，用管理文件的方式管理设备呢？答案是肯定的。以 UNIX 操作系统为例，它的核心设计思想之一是一切都是文件，设备也不例外。为了对各种设备进行统一管理，操作系统将设备按照各自的特点进行分类，组织为树状的层次结构挂载到设备文件系统（Device Filesystem, DevFS）下（默认为 `/dev` 目录）。设备文件系统的叶子节点称为设备文件（Device File），对应设备实例，其完整的路径名则作为设备的逻辑名称。设备文件是操作系统对物理设备抽象而成的虚拟设备，例如在 Linux 系统上，显卡的显存被抽象成帧缓存（Framebuffer，路径名为 `/dev/fb`），串口设备被抽象成终端（Teletype，路径名为 `/dev/tty`）。

如图 13.17 所示，有了设备文件系统，应用程序就可以直接使用文件系统的 `open` 接口打开设备文件，使用 `read/write` 接口读写设备文件的内容。通过复用文件的方式提供 I/O 设备的操作接口至少有三个优点：

- 操作系统的用户已经习惯了文件的使用方法，因此采用文件系统接口处理 I/O 更符合直觉。
- 设备名遵循文件名的用法和语义，设备文件系统由此可以向应用程序提供统一的设备命名空间和索引方式。
- 设备文件复用文件系统的权限管理，通过访问控制机制决定进程可以访问哪些设备。